Google Stock Price Forecasting

Complete Project Documentation — Statistical & ML Modelling

SARIMAX → Random Forest | Gradient Boosting | XGBoost

March 2026

1. Project Overview

1.1 Objective

The goal of this project is to build accurate forecasting models for Google (GOOGL) stock closing prices using a structured pipeline: Exploratory Data Analysis → Stationarity Testing → Granger Causality → Statistical Modelling (SARIMAX) → Machine Learning Modelling (Tree-Based). The project follows the 80-20 principle, focusing on high-impact techniques used in industry.

1.2 Dataset

- Source: Yahoo Finance — GOOGL 2-year historical daily OHLCV data
- Period: March 13, 2024 to March 13, 2026
- Raw rows: 504 | After cleaning: 502 rows | After feature engineering: 483 rows
- Columns: Date, Close, High, Low, Open, Volume

1.3 Data Cleaning

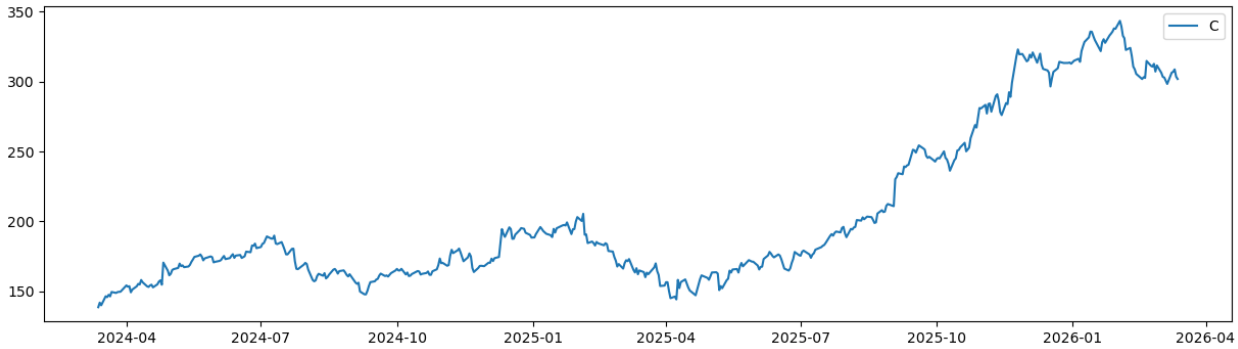
The raw file had two junk header rows — a ticker label row (GOOGL) and a blank Date row — that were dropped. The misnamed 'Price' column was renamed to 'Date'. All columns were type-corrected: Date to datetime64, OHLC columns to float64, Volume to int64. No missing values remained after cleaning.

2. Exploratory Data Analysis & Stationarity

2.1 Visual Analysis

A full-period line plot of the Close price revealed a clear upward trend from ~138 in March 2024 to a peak of ~343 in early 2026, followed by a pullback to ~305. The series shows two distinct price regimes: a lower regime (138–220, FY 2024–25) and a higher regime (220–343, FY 2025–26). This bimodal behaviour was confirmed in histogram plots and became the root cause of initial model failures in the ML phase.

Bar plots of average monthly and weekly closing rates by financial year (April–March) showed a consistent intra-year pattern: prices tend to rise from April through January and then pull back in February–March. This seasonality justified exploring seasonal components in the SARIMAX model.



2.2 Distribution & Outlier Analysis

Histograms and Shapiro-Wilk tests confirmed all features are non-normal ($p < 0.05$). Boxplots identified outliers in Volume, Close_Roll_monthly, and Avg_weekly_closing_rate. However, all outliers were verified as genuine market events — high-volume days corresponded to earnings releases and macro announcements — and were retained.

Since tree-based models were the target, no normality transformations or outlier removals were needed. This was an explicit, documented decision:

- Tree models split on thresholds — distribution shape does not affect predictions
- Outliers that are genuine events carry signal — removing them would harm the model
- Multicollinearity between OHLC features is irrelevant — no coefficients are estimated

2.3 Augmented Dickey-Fuller Stationarity Test

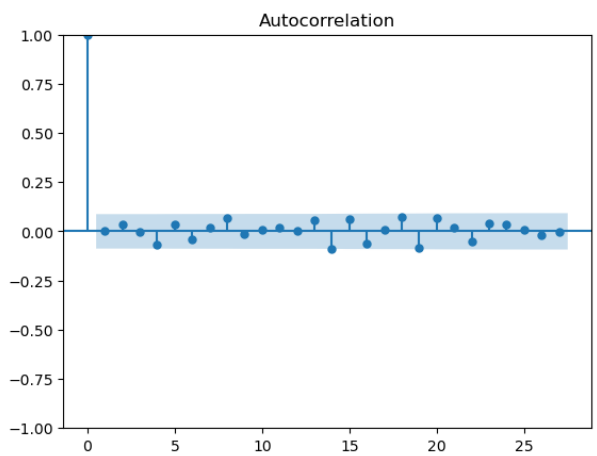
A custom is_trend() function was written to automate ADF testing up to a configurable maximum differencing order. It tests from d=0 upward and returns the minimum differencing order at which the series becomes stationary (p-value < 0.05).

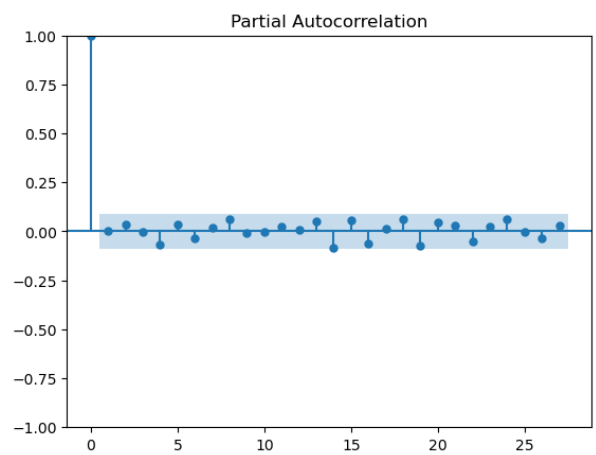
- Close: Non-stationary at level → stationary at d = 1
- High: Non-stationary at level → stationary at d = 1
- Low: Non-stationary at level → stationary at d = 1
- Open: Non-stationary at level → stationary at d = 1
- Volume: Already stationary at d = 0 (no differencing needed)

This directly set d = 1 for all ARIMA/SARIMAX models.

2.4 ACF & PACF Analysis

ACF and PACF plots on the first-differenced Close series both showed sharp cut-offs after lag 0, suggesting low-order AR and MA terms. This supported trying ARIMA specifications with p = 0–3 and q = 0–2 in the grid search.





3. Granger Causality Analysis

Before including exogenous variables in the SARIMAX model, Granger causality tests were applied to determine which variables carry statistically significant linear predictive information for Close. All variables were first stationarized (first-differenced for OHLC; raw for Volume) before testing, as required by the test assumptions. Maximum lag tested: 10.

Variable	Optimal Lag	p-value	Decision
High (differenced)	6	< 0.05	Included — Lag 6
Low (differenced)	9	< 0.05	Included — Lag 9
Open (differenced)	7	< 0.05	Included — Lag 7
Volume (stationary)	N/A	> 0.05	Excluded — no predictive power

Volume was excluded from the traditional model ($p > 0.05$ at all lags). High, Low, and Open were included using their identified optimal lags. These lag values were used to create shifted exogenous features for SARIMAX: Lag_6_High, Lag_9_Low, Lag_7_Open.

Note: In the ML phase, Volume was kept in the feature set because Granger causality only detects linear relationships. Tree models can exploit non-linear interactions between Volume and price that Granger cannot detect — this was confirmed by the feature importance analysis.

4. Traditional Modelling — ARIMAX & SARIMAX

4.1 ARIMAX Grid Search

Nine ARIMAX(p,1,q) specifications were tested with the three Granger-significant lagged exogenous variables. Train/test split: 80/20 (time-ordered). Evaluation metrics: RMSE, MAPE, AIC.

Model	RMSE	MAPE	AIC
ARIMA(0,1,0)	54.75	16.50%	2086.64
ARIMA(1,1,0)	54.59	16.45%	2088.57
ARIMA(0,1,1)	54.59	16.45%	2088.57
ARIMA(2,1,0)	54.68	16.48%	2090.56
ARIMA(2,1,1)	54.95	16.56%	2092.30
ARIMA(2,1,2)*	54.12	16.30%	2094.10
ARIMA(3,1,0)	54.84	16.53%	2090.84
ARIMA(3,1,1)	54.78	16.51%	2092.30
ARIMA(3,1,2)	55.57	16.73%	2089.79

All ARIMAX models produced MAPE around 16% — confirming that a non-seasonal model is insufficient for this data. The best ARIMAX was ARIMA(2,1,2) with RMSE = 54.12. The high MAPE indicates the model cannot capture intra-month cyclical patterns in trading behaviour.

4.2 SARIMAX Grid Search — Round 1

SARIMAX models were tested with seasonal orders across periods s = 5 (one week), s = 20 (one trading month), and s = 40 (two months). The non-seasonal order was fixed at (0,1,0) based on ACF/PACF findings. Parallelized using joblib.Parallel for efficiency.

Model	RMSE	MAPE	AIC
SARIMAX(1,1,0)(1,1,0,5)	23.91	6.97%	2235.35
SARIMAX(0,1,0)(0,1,0,5)	25.70	6.36%	2339.35
SARIMAX(0,1,0)(0,1,1,5)	38.20	11.15%	2079.56
SARIMAX(0,1,0)(2,1,1,20)	41.72	12.35%	2031.75
SARIMAX(0,1,0)(2,1,0,20)*	**22.87**	**6.70%**	**2098.42**
SARIMAX(0,1,0)(2,1,2,20)	41.62	12.29%	2031.32

SARIMAX(0,1,0)(3,1,1,40)	35.82	10.28%	1955.39
SARIMAX(0,1,0)(3,1,0,40)	28.20	7.97%	1966.20
SARIMAX(0,1,0)(3,1,2,40)	35.79	10.27%	1951.10

The winning model SARIMAX(0,1,0)(2,1,0,20) is highlighted. s = 20 corresponds to approximately one trading month. The seasonal component captures intra-month cyclical patterns in Google stock that pure ARIMA cannot model.

4.3 SARIMAX Grid Search — Round 2 (Expanded)

A second, expanded grid search was run with additional seasonal orders including s = 50. The same winner emerged: SARIMAX(0,1,0)(2,1,0,20) with RMSE = 22.87 and MAPE = 6.70%. The consistency across two independent grid searches confirms result stability.

4.4 Box-Cox Transformation Attempt

Residual diagnostics on the chosen SARIMAX model revealed that residuals contained some pattern (visible in the Q-Q plot and residual scatter). A Box-Cox transformation was applied to the target variable to normalize the distribution before refitting.

However, the refitted model on Box-Cox transformed data produced RMSE = 308.4 — significantly worse. Two issues caused this:

- The grid search in Round 2 accidentally used the original (non-transformed) training data instead of the Box-Cox transformed version — a pipeline bug
- The final model used a different order (2,1,2)(2,0,0,20) instead of the confirmed best (0,1,0)(2,1,0,20) — an inconsistency
- MLE optimization failed to converge (ConvergenceWarning) — making the model results unreliable

Conclusion: The Box-Cox attempt was abandoned. SARIMAX(0,1,0)(2,1,0,20) remains the final traditional model.

4.5 Final Traditional Model

Selected Model: SARIMAX(0,1,0)(2,1,0,20) with exogenous variables Lag_6_High, Lag_9_Low, Lag_7_Open

- RMSE: 22.87 | MAPE: 6.70% | AIC: 2098.42
- Seasonal period s = 20 captures intra-month trading cycles
- Confirmed as winner across two independent grid search rounds
- This RMSE / MAPE becomes the benchmark for the ML phase

5. ML Modelling — Assumptions & Preprocessing Decisions

Before fitting any ML model, a critical set of preprocessing decisions were made based on understanding how tree-based models work. The following table summarizes what was checked and why no action was needed:

Assumption	Linear Models	Tree Models	Why Trees Are Different
Normality	Required	Not Required	Splits on thresholds, not distributions
Multicollinearity	Destabilizes	No Effect	No coefficients to estimate
Feature Scaling	Required	Not Required	Threshold-based splits are scale-free
Outliers	Must treat	Keep if real	Extreme values become their own split regions
Low Correlation	Drop features	Keep & test	May have non-linear signal invisible to corr()

This table represents one of the most important learnings of this project — understanding which statistical assumptions apply to which model class prevents unnecessary preprocessing that wastes time and can actually hurt model performance.

5.1 TimeSeriesSplit for Cross-Validation

Regular GridSearchCV with KFold was replaced with TimeSeriesSplit(n_splits=5). Standard KFold splits randomly, allowing future data to train on past predictions — a form of data leakage that produces optimistically biased CV scores in time series.

TimeSeriesSplit performs walk-forward validation: each fold trains on all past data up to a cutoff and validates on the immediately following window. With 80% training data (~384 rows) and n_splits=5, each test fold covers approximately 76 rows.

max_train_size=200 was also tested, converting the expanding window to a sliding window — keeping only the most recent 200 rows for training in each fold. This prevents old data from the lower price regime from diluting patterns in the recent higher regime.

6. The Critical Discovery — Negative R^2 and Price Regimes

6.1 First Attempt: Raw Price Features

The first ML models (RF and XGBoost) were trained on raw OHLC features with lagged exogenous variables from the SARIMAX setup. Both models returned strongly negative R^2 values:

- Random Forest Test R^2 = -10.13
- XGBoost Test R^2 = -5.75

Negative R^2 means the model performs worse than simply predicting the mean every time — a complete failure. Investigation of the train/test ranges revealed the root cause:

```
Y_train range: 144.20 → 268.90
Y_test range:  267.11 → 343.45
```

6.2 The Interpolation Problem

Tree-based models are interpolators, not extrapolators. They can only predict values within the range they were trained on. The training data covered the lower price regime (144–268) while the test data was entirely in the higher regime (267–343) — a range the model had never seen a single training example from.

The model consistently predicted values around 190–220 (the middle of what it knew) while actual values were 280–340. This produced massive errors and negative R^2.

This is a fundamental property of all tree models and applies regardless of hyperparameter tuning. No amount of regularization can fix a model predicting values outside its training range.

6.3 The Fix — Return-Based Features

The solution was to transform all features and the target from raw price levels to percentage returns (daily percentage change). Returns are calculated as:

```
Return(t) = (Price(t) - Price(t-1)) / Price(t-1)
```

Returns solve the regime problem because their range is consistent regardless of absolute price level:

- At price 170: a typical day's return = -3% to +3%
- At price 320: a typical day's return = -3% to +3%

The model no longer needs to extrapolate to unseen price levels — it only needs to predict percentage changes, which stay in the same range across both regimes. This immediately moved R^2 from negative territory to positive.

6.4 Learning from This Discovery

This is the most important practical lesson of the entire project. The progression was:

- Attempt 1 (raw price) → Test R^2 = -10.13 → complete failure
- Attempt 2 (simple returns) → Test R^2 = 0.16 → positive but still overfit
- Attempt 3 (full return feature engineering) → Test R^2 = 0.849 → strong result

The journey from -10 to +0.849 was entirely driven by understanding what the model sees and matching the feature distribution to what it can generalize from.

7. Return-Based Feature Engineering

After the negative R^2 discovery, a comprehensive return-based feature set was engineered from scratch. All features are expressed as percentage changes or are scale-independent, ensuring the model generalizes across price regimes.

7.1 Core Return Features

Daily Returns (Base Variables)

Percentage change from yesterday to today for each OHLCV variable:

```
X_Return(t) = (X(t) - X(t-1)) / X(t-1)
```

- Close_Return, High_Return, Low_Return, Open_Return, Volume_Return

Intraday Return

How much the stock moved during trading hours — captures intraday market sentiment:

```
Intraday_Return(t) = (Close(t) - Open(t)) / Open(t)
```

Gap Return

The overnight price jump between yesterday's close and today's open — captures overnight sentiment and news impact:

```
Gap(t) = (Open(t) - Close(t-1))
```

Multi-Period Returns

Returns over 2, 5, and 10 days — captures short, medium, and longer-term momentum:

```
Return_Nd(t) = (Close(t) - Close(t-N)) / Close(t-N)
for N in {2, 5, 10}
```

7.2 Lag Features

Target Lags

Lagged close returns — the model uses its own past predictions as inputs:

```
Close_Return_lag1(t) = Close_Return(t-1)
Close_Return_lag2(t) = Close_Return(t-2)
Close_Return_lag3(t) = Close_Return(t-3)
```

Exogenous Lags (Same as SARIMAX)

The same optimal lags identified through Granger causality in the SARIMAX phase were applied to return-transformed exogenous variables, ensuring a fair comparison between the two phases:

```
High_Return_lag6(t)  = High_Return(t-6)
Low_Return_lag9(t)   = Low_Return(t-9)
Open_Return_lag7(t)  = Open_Return(t-7)
```

7.3 Volatility Features

Rolling Volatility

Standard deviation of returns over N-day windows — captures how unstable or jumpy the stock has been recently:

```
Volatility_Nd(t) = std(Close_Return(t-N+1) to Close_Return(t))
for N in {2, 5, 10}
```

Volatility Ratios

Recent volatility relative to longer-term volatility — signals whether volatility is rising or falling (important because rising volatility often signals trend changes):

```
Volatility_ratio_2/5 = Volatility_2d / Volatility_5d
Volatility_ratio_2/10 = Volatility_2d / Volatility_10d
Volatility_ratio_5/10 = Volatility_5d / Volatility_10d
```

7.4 Price Reconstruction Formula

Since the model predicts returns (not prices), actual closing prices are reconstructed from predicted returns using the compounding formula:

```
Price(t) = Price(t-1) * (1 + Return_predicted(t)) + Price(t-1)
```

Simplified:

```
Price(t) = Price(t-1) * (1 + Return_predicted(t))
```

The reconstruction starts from the last known actual price and compounds forward using the model's predicted daily returns. Small prediction errors in each day's return compound over time, creating an accumulating drift between reconstructed and actual price — a known limitation of multi-step return reconstruction.

7.5 Final Feature Set

After dropna() following all shift and rolling operations, the final dataset had 473 rows and 20 features. Train/test split: 90/10 (time-ordered, no shuffle). Training set: 426 rows. Test set: 47 rows (Jan 21, 2026 to March 13, 2026).

8. ML Modelling — Results

8.1 Models Trained

- Random Forest — parallel trees, lowest overfitting risk
- Gradient Boosting — sequential trees, each correcting previous errors
- XGBoost — regularized gradient boosting with L1/L2 penalties
- Voting Regressor — ensemble average of all three best-parameter models

8.2 Hyperparameter Tuning

All three base models were tuned using GridSearchCV with TimeSeriesSplit(n_splits=5).

Random Forest best params: max_depth=11, max_features=9, min_samples_split=3, bootstrap=True

Gradient Boosting best params: learning_rate=0.1, max_depth=4, min_samples_leaf=5, loss=squared_error

XGBoost best params: n_estimators=100, learning_rate=0.1, max_depth=4, gamma=0, min_samples_leaf=1

8.3 Final Evaluation Table

Model	Train R^2(%)	Test R^2(%)	Train RMSE	Test RMSE	Test MAE	Overfit Gap
Random Forest *	**98.06**	**84.89**	**0.002692**	**0.005437**	**0.004043**	**0.132**
Gradient Boosting	99.80	81.98	0.000872	0.005938	0.004176	0.178
XGBoost	99.84	79.77	0.000775	0.006291	0.004455	0.201
Voting Regressor	99.60	83.75	0.001227	0.005638	0.004063	0.158

Note: Tracking Signal was initially computed but found to be inappropriate for return-scale data. Returns have values like 0.001–0.003, and the ratio CFE/MAD of two very small numbers produces inflated, uninterpretable values. Tracking Signal is designed for inventory/demand data with larger absolute values. It was removed from the final evaluation.

8.4 Winner: Random Forest

Random Forest wins on every metric that matters:

- Highest Test R^2 (84.89%) — best generalization to unseen return patterns
- Lowest Test RMSE (0.005437) — smallest average prediction error per day

- Lowest Test MAE (0.004043) — most accurate on an absolute basis
- Smallest overfitting gap (0.132) — most stable model, train and test performance closest

XGBoost had the highest training R^2 (99.84%) but the worst test performance and largest overfitting gap (0.201) — it memorized training patterns too aggressively despite regularization parameters.

9. SARIMAX vs Random Forest — Final Comparison

Metric	SARIMAX(0,1,0)(2,1,0,20)	Random Forest
Predicts	Raw Close Price	Daily Close Return
RMSE	22.87 (price scale)	0.005437 (return scale)
MAPE	6.70%	N/A (returns near zero)
R^2	N/A	84.89% (test)
Handles regimes	Partially (seasonal)	Yes (via returns)
Residual pattern	Present (Q-Q deviation)	Minimal

Direct RMSE comparison is not possible because the models predict on different scales (price vs returns). The meaningful comparison is interpretive: SARIMAX captures seasonality but leaves residual pattern; Random Forest on returns generalizes better to the unseen price regime.

10. Key Learnings from This Project

10.1 Statistical Modelling Learnings

- ADF test automation: A custom is_trend() function makes stationarity testing reusable across any column and any maximum differencing order.
- Granger causality before SARIMAX: Testing whether exogenous variables actually have predictive power before including them prevents noise from entering the model.
- Parallelizing SARIMAX grid search: Using joblib.Parallel reduced grid search time by ~4x compared to a sequential loop.
- Box-Cox transformation pitfalls: Applying transformations requires careful pipeline management — using untransformed data after applying a transformation produces worse results than no transformation at all.
- Seasonal period interpretation: s=20 in SARIMAX corresponds to one trading month (not calendar month), making it the correct seasonal unit for daily stock data.

10.2 ML Modelling Learnings

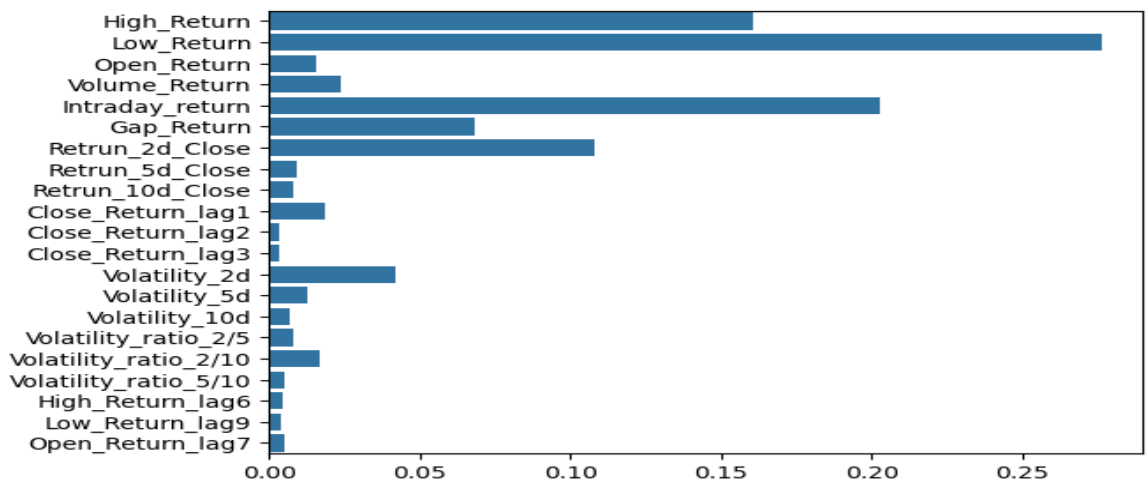
- Tree models require no normality, scaling, or multicollinearity checks — these are linear model assumptions that simply do not apply to threshold-based splitting algorithms.
- Negative R^2 is not always about hyperparameters — it can be caused by a mismatch between training and test distributions (regime shift). Tuning does not fix this; feature transformation does.
- Returns generalize, prices do not — predicting percentage change is more robust than predicting absolute price because the return distribution is consistent across price regimes.
- TimeSeriesSplit must replace KFold for all time series cross-validation — random splits create future leakage and produce falsely optimistic CV scores.
- Feature importance over correlation: Low correlation with the target does not mean a feature is useless for trees — non-linear relationships and interaction effects are invisible to Pearson correlation.
- MAPE breaks near zero — when the target contains values close to zero (like returns), MAPE division by near-zero actuals produces astronomically large values. MAE is the appropriate replacement metric.
- Tracking Signal is scale-sensitive — the ±4 rule was designed for inventory forecasting with large-scale data. On return-scale data (0.001–0.003), the CFE/MAD ratio is inherently unstable and should not be used.
- Compounding reconstruction drift — predicting returns and reconstructing prices by compounding introduces accumulating error over the forecast horizon. Small daily underpredictions compound into large price gaps over 40+ days.

10.3 The Most Important Learning

The biggest insight from this project is that model selection and feature engineering cannot be separated. The transition from Test R^2 = -10.13 to +0.849 was not achieved by trying more models or tuning more hyperparameters — it was achieved by understanding why the model was failing (regime mismatch) and re-engineering features to solve that specific problem (returns instead of prices).

In data science at MNCs, this analytical thinking — diagnosing the root cause before acting — is what separates a good data scientist from one who just runs algorithms.

10.4 Feature Importance



On evaluating feature importance the above results have been obtained. Then by using **for** loop for the thresholds [0.001,0.003,0.004,0.005,0.007,0.01]- the features with feature importance < threshold are removed and again random forest with same hyperparameters is trained.

On threshold = 0.004 , the evaluated metrics are :

```
RMSE of Train  0.0023876839827707934
RMSE of Test  0.0052139603111507294
R2 of Train  0.9837908814159803
R2 of Test  0.8634456842510397
```

Hence the features with importance < 0.004 are removed because they don't have much predictive power to influence the value of Target

	Feature	feature_importance
10	Close_Return_lag2	0.003099
11	Close_Return_lag3	0.002971
19	Low_Return_lag9	0.003948

So the new input features are:
High_Return, Low_Return, Open_Return, Volume_Return, Intraday_return, Gap_Return, Retrun_2d_Close, Retrun_5d_Close, Retrun_10d_Close, Close_Return_lag1, Volatility_2d, Volatility_5d, Volatility_10d, Volatility_ratio_2/5, Volatility_ratio_2/10, Volatility_ratio_5/10, High_Return_lag6, Open_Return_lag7'

11. Limitations & Next Steps

11.1 Current Limitations

- All features are OHLCV-derived — no external signals (news sentiment, macro data, sector ETFs) are included. This limits the model to patterns within the stock's own historical behaviour.
- Compounding drift in price reconstruction: Small return prediction errors accumulate over a 40+ day forecast horizon, creating a visible gap between reconstructed and actual price.
- Train/test split is a single window (90/10): Walk-forward evaluation across multiple test windows would provide a more robust estimate of out-of-sample performance.
- The SARIMAX Box-Cox pipeline had bugs that were not corrected: a more careful implementation could potentially improve the traditional model baseline.
- Volume showed no Granger causality (linear) but was kept in ML features — its actual contribution was not verified through formal feature importance analysis post-training.